SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively.*(BL3)*

Assessment Tool Weightage: 50%

QUESTIONS: 3 **Duration : 1 Hour**
Total Marks:30

Annexure A

Constraint-Based Problem Solving

Code of Conduct:

I Mukesh Raj L(192571036) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

Constraint-Based Problem Solving

1. A hospital needs to assign 3 doctors (D1, D2, D3) to 3 shifts (Morning, Afternoon, Night) such that:

   Constraints:

   i. D1 cannot work Night shift
   ii. D2 must work before D3 (Morning < Afternoon < Night)
   iii. Only one doctor per shift
   iv. D3 cannot work Morning
   v. All doctors must be assigned exactly one shift

   Apply Backtracking Search to find a valid assignment with all intermediate steps and constraint checks. State the final valid schedule

2. A robot operates in a grid environment (5×5). It must move from Start (S) to Goal (G). Obstacles block certain paths.

   Constraints:

   i. Movement allowed: Up, Down, Left, Right
   ii. Cost of each move = 1
   iii. Cannot pass through obstacles
   iv. Use Manhattan Distance heuristic

   Using above constraints and BFS Search Algorithm Show step-by-step node expansion and priority queue updates and determine the optimal path and cost.

3. An autonomous rescue robot is deployed in a disaster-affected building to locate and rescue survivors. The robot operates in a partially observable environment where some paths are blocked, and it must make decisions with limited information.

The building is represented as a grid of rooms. The robot starts at position S and must reach a survivor located at G.

Constraints:

- The robot can move up, down, left, right
- Some cells are blocked (obstacles) and cannot be traversed
- The robot has limited sensing capability (can only observe adjacent cells)
- Each movement has a cost of 1, but entering risky zones has an additional cost of 2
- The robot must minimize total cost while ensuring safe navigation
- The robot must update its knowledge dynamically (online search scenario)

Tasks:

a) Analyze all the given constraints and explain how they affect the robot's decision-making process.

b) Develop a solution approach using an appropriate search strategy (e.g., Uniform Cost Search / Online Search Agent). Clearly explain the steps involved.
c) Demonstrate how the robot applies AI concepts such as:

- Intelligent agents
- Rationality
- Environment type

d) Justify why your chosen approach is the most suitable for solving this problem under the given constraints.

1)
Problem Statement

A hospital needs to assign three doctors (D1, D2, D3) to three shifts (Morning, Afternoon, Night) such that all the given constraints are satisfied.

Constraints

1. D1 cannot work the Night shift.
2. D2 must work before D3 (Morning < Afternoon < Night).
3. Only one doctor can be assigned to each shift.
4. D3 cannot work the Morning shift.
5. Every doctor must be assigned exactly one shift.

Backtracking Search

Backtracking is a depth-first search algorithm used to solve CSPs.

It works by

1. Assigning a value.
2. Checking constraints.
3. If constraints fail, undo the assignment.
4. Try another value.
5. Continue until a solution is found.

Solution

The Backtracking Search Algorithm starts by assigning shifts to doctors one at a time. After each assignment, it checks whether all constraints are satisfied. If any constraint is violated, the algorithm removes the current assignment (backtracks) and tries another shift. This process continues until a valid schedule is obtained.

Python Code

```
class HospitalScheduler:

    def __init__(self):

        self.doctors = ["D1", "D2", "D3"]

        self.shifts = ["Morning", "Afternoon", "Night"]

        self.assignment = {}

    def check_constraints(self, doctor, shift):

        # D1 cannot work Night

        if doctor == "D1" and shift == "Night":

            return False

        # D3 cannot work Morning
```

```
    if doctor == "D3" and shift == "Morning":
        return False
    # Only one doctor per shift
    if shift in self.assignment.values():
        return False
    # D2 must work before D3
    if doctor == "D3" and "D2" in self.assignment:
        if self.shifts.index(self.assignment["D2"]) >= self.shifts.index(shift):
            return False
    if doctor == "D2" and "D3" in self.assignment:
        if self.shifts.index(shift) >= self.shifts.index(self.assignment["D3"]):
            return False
    return True
def display_assignment(self):
    print("\nCurrent Assignment")
    for doctor, shift in self.assignment.items():
        print(f"{doctor} --> {shift}")
def backtracking_search(self, doctor_index):
    if doctor_index == len(self.doctors):
        return True
    current_doctor = self.doctors[doctor_index]
    print(f"\nAssigning Shift for {current_doctor}")
    for shift in self.shifts:
        print(f"Trying {current_doctor} -> {shift}")
        if self.check_constraints(current_doctor, shift):
            print("Constraint Satisfied")
            self.assignment[current_doctor] = shift
```

```
            self.display_assignment()
            if self.backtracking_search(doctor_index + 1):
                return True
            print(f"Backtracking from {current_doctor} -> {shift}")
            del self.assignment[current_doctor]
        else:
            print("Constraint Violated")
    return False
  def display_final_schedule(self):
    print("\nFinal Schedule")
    print("-" * 30)
    print("{:<10}{}".format("Doctor", "Shift"))
    print("-" * 30)
    for doctor in self.doctors:
        print("{:<10}{}".format(doctor, self.assignment[doctor]))
    print("-" * 30)
  def solve(self):
    print("Hospital Doctor Shift Scheduling")
    print("Backtracking Search")
    if self.backtracking_search(0):
        print("\nSolution Found")
        self.display_final_schedule()
    else:
        print("\nNo Valid Solution Exists")
def main():
  scheduler = HospitalScheduler()
  scheduler.solve()
```

```
if __name__ == "__main__":
    main()
```

Output

```
Current Assignment
D1 --> Morning
D2 --> Afternoon

Assigning Shift for D3
Trying D3 -> Morning
Constraint Violated
Trying D3 -> Afternoon
Constraint Violated
Trying D3 -> Night
Constraint Satisfied
```

```
Assigning Shift for D1
Trying D1 -> Morning
Constraint Satisfied

Current Assignment
D1 --> Morning

Assigning Shift for D2
Trying D2 -> Morning
Constraint Violated
Trying D2 -> Afternoon
Constraint Satisfied
```

```
Current Assignment
D1 --> Morning
D2 --> Afternoon
D3 --> Night

Solution Found
```

```
Final Schedule
------------------------------
Doctor    Shift
------------------------------
D1        Morning
D2        Afternoon
D3        Night
------------------------------
```

Report

The hospital shift scheduling problem was successfully solved using the Backtracking Search Algorithm. The algorithm efficiently checked all constraints, avoided invalid assignments through backtracking, and generated a valid schedule where each doctor was assigned exactly one shift. This demonstrates that Backtracking Search is an effective technique for solving Constraint Satisfaction Problems (CSPs).

2)

Problem Statement

A robot operates in a 5×5 grid environment where it must travel from the Start (S) position to the Goal (G) position. Some cells contain obstacles that block movement. The robot is allowed to move only in four directions: Up, Down, Left, and Right. Each movement has a cost of 1, and the Manhattan Distance heuristic is used to select the next node with the highest priority. The objective is to determine an optimal path from the start to the goal while avoiding obstacles.

Solution

The Greedy Best-First Search (GBFS) algorithm selects the next node based on the minimum Manhattan Distance to the goal. It explores valid neighboring nodes, avoids obstacles, and continues expanding nodes until the goal is reached. The algorithm successfully determines the optimal path from the Start node to the Goal node.

Python Code:

```
import heapq

class GreedyBestFirstSearch:

    def __init__(self):

        self.grid = [

            ['S', 0, 0, 'X', 0],

            [0, 'X', 0, 'X', 0],

            [0, 'X', 0, 0, 0],

            [0, 0, 'X', 'X', 0],

            [0, 0, 0, 'G', 0]

        ]

        self.rows = len(self.grid)

        self.cols = len(self.grid[0])

        self.start = (0, 0)
```

```
    self.goal = (4, 3)
    self.moves = [
        (-1, 0),
        (1, 0),
        (0, -1),
        (0, 1)
    ]
def manhattan_distance(self, current):
    return abs(current[0] - self.goal[0]) + abs(current[1] - self.goal[1])
def is_valid(self, row, col):
    return (
        0 <= row < self.rows and
        0 <= col < self.cols and
        self.grid[row][col] != 'X'
    )
def display_path(self, path):
    print("\nOptimal Path")
    for node in path:
        print(node, end=" ")
    print()
def greedy_search(self):
    priority_queue = []
    heapq.heappush(
        priority_queue,
        (self.manhattan_distance(self.start), self.start)
    )
    visited = set()
```

```
parent = {}
print("Node Expansion\n")
while priority_queue:
    heuristic, current = heapq.heappop(priority_queue)
    if current in visited:
        continue
    visited.add(current)
    print("Expanded Node :", current)
    print("Heuristic Value :", heuristic)
    if current == self.goal:
        break
    for move in self.moves:

        new_row = current[0] + move[0]
        new_col = current[1] + move[1]
        if self.is_valid(new_row, new_col):
            neighbour = (new_row, new_col)
            if neighbour not in visited:
                parent[neighbour] = current
                heapq.heappush(
                    priority_queue,
                    (
                        self.manhattan_distance(neighbour),
                        neighbour
                    )
                )
    print("Priority Queue :", priority_queue)
```

```
            print()
        path = []
        current = self.goal
        while current != self.start:
            path.append(current)
            current = parent[current]
        path.append(self.start)
        path.reverse()
        self.display_path(path)
        print("\nTotal Cost :", len(path) - 1)
def main():
    search = GreedyBestFirstSearch()


    search.greedy_search()
if __name__ == "__main__":
    main()
```

Output

```
Expanded Node : (0, 0)
Heuristic Value : 7
Priority Queue : [(6, (0, 1)), (6, (1, 0))]

Expanded Node : (0, 1)
Heuristic Value : 6
Priority Queue : [(5, (0, 2)), (6, (1, 0))]

Expanded Node : (0, 2)
Heuristic Value : 5
Priority Queue : [(4, (1, 2)), (6, (1, 0))]
```

```
Expanded Node : (1, 2)
Heuristic Value : 4
Priority Queue : [(3, (2, 2)), (6, (1, 0))]

Expanded Node : (2, 2)
Heuristic Value : 3
Priority Queue : [(2, (2, 3)), (6, (1, 0))]

Expanded Node : (2, 3)
Heuristic Value : 2
Priority Queue : [(3, (2, 4)), (6, (1, 0))]
```

```
Expanded Node : (2, 4)
Heuristic Value : 3
Priority Queue : [(2, (3, 4)), (6, (1, 0)), (4, (1, 4))]

Expanded Node : (3, 4)
Heuristic Value : 2
Priority Queue : [(1, (4, 4)), (6, (1, 0)), (4, (1, 4))]

Expanded Node : (4, 4)
Heuristic Value : 1
Priority Queue : [(0, (4, 3)), (6, (1, 0)), (4, (1, 4))]
```

```
Expanded Node : (4, 3)
Heuristic Value : 0

Optimal Path
(0, 0) (0, 1) (0, 2) (1, 2) (2, 2) (2, 3) (2, 4) (3, 4) (4, 4) (4, 3)

Total Cost : 9

Process finished with exit code 0
```

Report

The robot pathfinding problem was successfully solved using the Greedy Best-First Search (GBFS) algorithm with the Manhattan Distance heuristic. The algorithm efficiently navigated the robot around obstacles and reached the goal by expanding the most promising nodes. The optimal path was found with a total cost of 9.

3)

Problem Statement

An autonomous rescue robot is deployed in a disaster-affected building to locate and rescue survivors. The robot starts from the Start (S) position and must reach the Goal (G) while avoiding blocked paths and risky zones. The robot has limited sensing capability and updates its knowledge dynamically. The objective is to minimize the total cost and safely reach the survivor using an appropriate AI search strategy.

Solution

a) Constraint Analysis

Constraint	Effect on Decision Making
Movement only in four directions	Limits the robot's possible actions.
Obstacles cannot be crossed	The robot must find an alternative path.
Limited sensing capability	The robot knows only adjacent cells and explores gradually.
Movement cost = 1	The robot calculates the total travel cost.
Risky zone cost = 2	The robot prefers safer paths with lower total cost.
Dynamic knowledge update	The robot continuously updates its map as new information becomes available.

b) Solution Approach (Uniform Cost Search with Online Search)

1. Initialize the robot at the Start position.
2. Observe adjacent cells using the robot's sensors.
3. Identify safe and reachable neighboring cells.
4. Calculate the movement cost for each path.
5. Use **Uniform Cost Search (UCS)** to select the path with the minimum accumulated cost.
6. Update the map whenever new obstacles or risky zones are detected.
7. Repeat the search until the Goal (survivor) is reached.
8. Display the optimal path and total cost.

c) AI Concepts Applied

Intelligent Agent

The rescue robot acts as an intelligent agent by sensing the environment, processing information, and taking appropriate actions to reach the survivor safely.

Rationality

The robot always chooses the path with the minimum total cost while avoiding obstacles and risky areas.

Environment Type

- **Partially Observable:** The robot can observe only adjacent cells.
- **Dynamic:** The environment changes as the robot explores.
- **Discrete:** The building is represented as grid cells.
- **Sequential:** Each decision affects future movements.

d) Justification

Uniform Cost Search (UCS) combined with an **Online Search Agent** is the most suitable approach because it always selects the least-cost path while dynamically updating information about the environment. This allows the robot to avoid blocked paths, minimize travel cost, and safely navigate toward the survivor even with limited knowledge.

Python Code

```
import heapq

class RescueRobot:

    def __init__(self):

        self.grid = [

            ['S', 0, 0, 'X', 0],

            [0, 'R', 0, 'X', 0],

            [0, 0, 0, 'R', 0],

            ['X', 0, 0, 0, 0],

            [0, 0, 'X', 'G', 0]

        ]

        self.start = (0, 0)

        self.goal = (4, 3)

        self.rows = len(self.grid)

        self.cols = len(self.grid[0])

        self.directions = [

            (-1, 0),

            (1, 0),

            (0, -1),

            (0, 1)

        ]

    def valid_position(self, row, col):

        return (

            0 <= row < self.rows and

            0 <= col < self.cols and
```

```
        self.grid[row][col] != 'X'
    )
def movement_cost(self, row, col):
    if self.grid[row][col] == 'R':
        return 3
    return 1
def observe_adjacent(self, current):
    print("\nObserved Adjacent Cells")
    row, col = current
    for dr, dc in self.directions:
        nr = row + dr
        nc = col + dc
        if self.valid_position(nr, nc):
            print((nr, nc), ":", self.grid[nr][nc])
def reconstruct_path(self, parent):
    path = []
    current = self.goal
    while current != self.start:
        path.append(current)
        current = parent[current]
    path.append(self.start)
    path.reverse()
    return path
def uniform_cost_search(self):
    priority_queue = []
    heapq.heappush(priority_queue, (0, self.start))
    visited = set()
```

```
parent = {}
total_cost = {
    self.start: 0
}
print("\nUNIFORM COST SEARCH\n")
while priority_queue:
    cost, current = heapq.heappop(priority_queue)
    if current in visited:
        continue
    visited.add(current)
    print("--------------------------------------")
    print("Current Node :", current)
    print("Current Cost :", cost)
    self.observe_adjacent(current)
    if current == self.goal:
        print("\nGoal Reached")
        break
    row, col = current
    for dr, dc in self.directions:
        nr = row + dr
        nc = col + dc
        if self.valid_position(nr, nc):
            neighbour = (nr, nc)
            new_cost = cost + self.movement_cost(nr, nc)
            if neighbour not in total_cost or new_cost < total_cost[neighbour]:
                total_cost[neighbour] = new_cost
                parent[neighbour] = current
```

```
                heapq.heappush(priority_queue, (new_cost, neighbour))
            print("\nPriority Queue")
            print(priority_queue)
        path = self.reconstruct_path(parent)
        print("\nOptimal Path")
        for node in path:
            print(node, end=" ")
        print("\n")
        print("Minimum Cost :", total_cost[self.goal])
def main():
    robot = RescueRobot()
    robot.uniform_cost_search()
if __name__ == "__main__":
    main()
```

Output